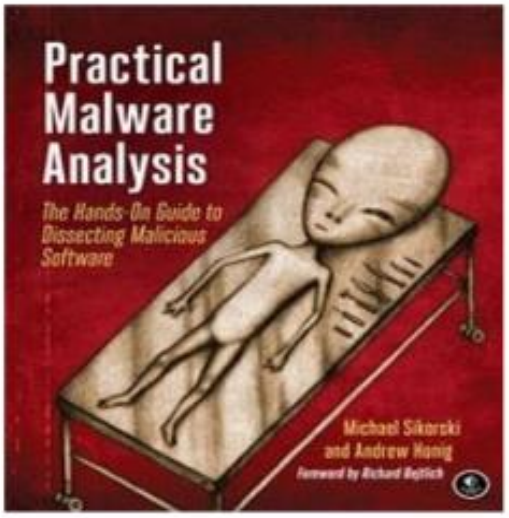
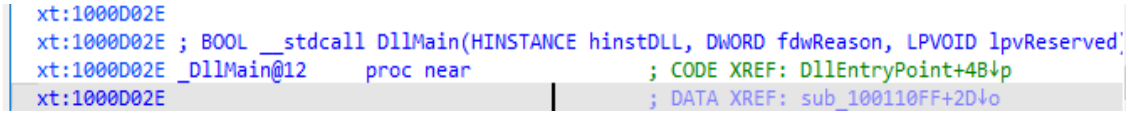
PMA lab 5 Writeup





https://www.linkedin.com/in/talha-yaqoob

1. **What is the address of DllMain?**

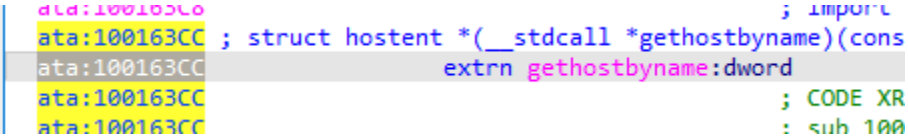


this is the entry point of DLL main

1000D02E

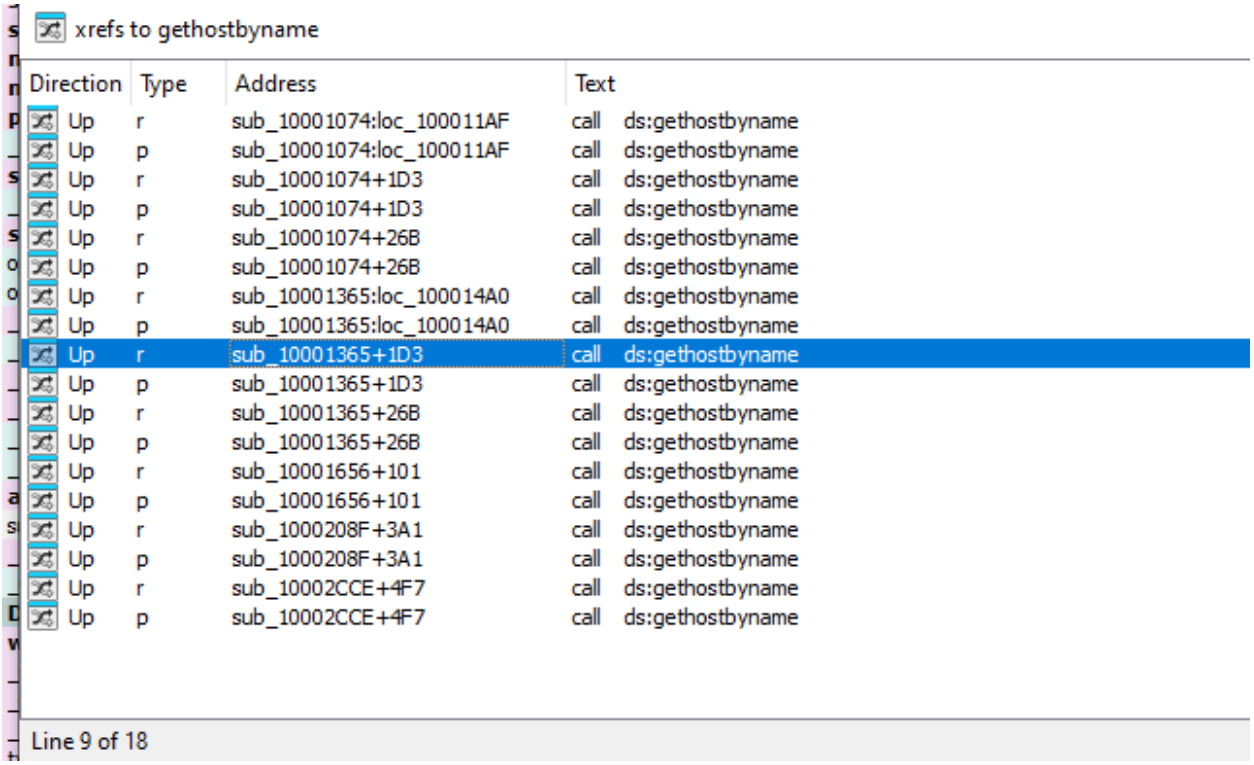
DllMain at 0x1000D02E serves as the initial execution point when the DLL is loaded into a process. From here, the malware begins setup activities such as initializing global state and invoking additional routines that later perform network communication, environment checks, and command execution. This indicates the malware relies on implicit execution rather than user interaction

2. Use the Imports window to browse to gethostbyname. Where is the import located?



the address is 100163CC

How many functions call gethostbyname?



called 9 times by 5 different functions

The malware imports and repeatedly calls `gethostbyname`, indicating it resolves a hardcoded domain name rather than using a static IP address.

Focusing on the call to gethostbyname located at 0x10001757, can you fig ure out which DNS request will be made?

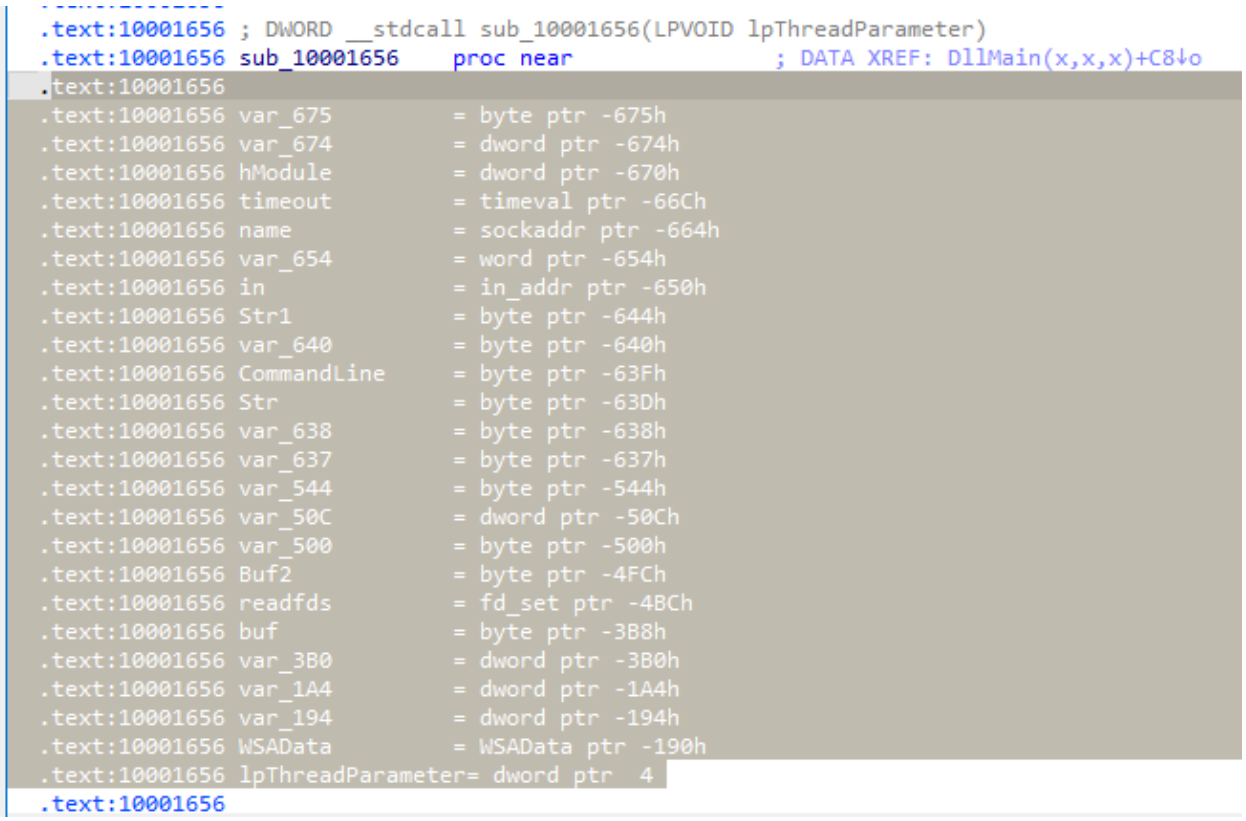
```
.7AD                    call    esi ; strncpy
.7AF                    add     esp, 18h
.7B2                    push    ebx             ; uCmdShow
.7B3                    push    offset CmdLine  ; "ipconfig /flushdns"
.7B8                    call    ds:WinExec
```

intents to flush dns cache to call the following domain

```
.7E8                    call    esi ; strncpy
.7EA                    add     esp, 18h
.7ED
.7ED loc_100017ED:                              ; CODE XREF: sub_10001656+E6↑j
.7ED                                            ; sub_10001656+F2↑j ...
.7ED                    mov     eax, off_10019040 ; "[This is RDO]pics.praticalmalwareanalys".
.7F2                    push    10h             ; Size
```

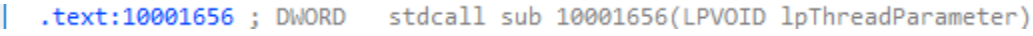
pics.praticalmalwareanalys…

How many local variables has IDA Pro recognized for the subroutine at 0x10001656?



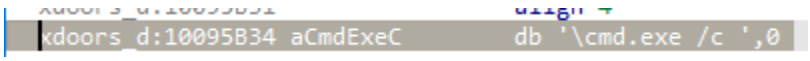
24 to be exact

How many parameters has IDA Pro recognized for the subroutine at 0x10001656?



only 1

Use the Strings window to locate the string \cmd.exe /c in the disassembly. Where is it located?



it is located at 1009FB34

What is happening in the area of code that references \cmd.exe /c

We have a string

```
.B31                    align 4
.B34 aCmdExeC           db '\cmd.exe /c ',0     ; DATA XREF: sub_1000FF58+278↑o
.B41                    align 4
.B44 ; char aHiMasterDDDDDD[]
.B44 aHiMasterDDDDDD db 'Hi,Master [%d/%d/%d %d:%d:%d]',0Dh,0Ah
.B44                                            ; DATA XREF: sub_1000FF58+145↑o
.B63                    db 'WelCome Back...Are You Enjoying Today?',0Dh,0Ah
.B8B                    db 0Dh,0Ah
.B8D                    db 'Machine UpTime  [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Secon'
.BCE                    db 'ds]',0Dh,0Ah
.BD3                    db 'Machine IdleTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Seco'
.C14                    db 'nds]',0Dh,0Ah
.C1A                    db 0Dh,0Ah
.C1C                    db 'Encrypt Magic Number For This Remote Shell Session [0x%02x]',0Dh,0Ah
.C59                    db 0Dh,0Ah,0
.C5C ; char asc_10095C5C[]
.C5C asc_10095C5C       db '>',0                ; DATA XREF: sub_1000FF58+4B↑o
.C5C                                            ; sub_1000FF58+3E1↑o
.C5E                    align 400h
```

welcoming it's master likely the attacker

The /c flag allows execution of attacker-supplied commands while immediately terminating the shell the surrounding code gathers the machine up time as well as machine idle time

then we have the "Encrpyt magic " which encrypts the shell session

In the same area, at 0x100101C8, it looks like dword_1008E5C4 is a global variable that helps decide which path to take. How does the malware set dword_1008E5C4? (Hint: Use dword_1008E5C4's cross-references.)

xrefs to dword_1008E5C4

Directio	Type	Address	Text	
Up	w	sub_10001656+22	mov	dword_1008E5C4, eax
Up	r	sub_10007312+E	cmp	dword_1008E5C4, edi
	r	sub_1000FF58+270	cmp	dword_1008E5C4, ebx

the global variable is set to be based on the value of the eax register which is later compared twice which indicates it is a condition for further execution.

A few hundred lines into the subroutine at 0x1000FF58, a series of com parisons use memcmp to compare strings. What happens if the string compar ison to robotwork is successful (when memcmp returns 0)?

```
)010446                 lea     eax, [ebp+Buf1]
)01044C                 push    offset aRobotwork ; "robotwork"
)010451                 push    eax             ; Buf1
)010452                 call    memcmp
)010457                 add     esp, 0Ch
)01045A                 test    eax, eax
)01045C                 jnz     short loc_10010468
)01045E                 push    [ebp+s]         ; s
)010461                 call    sub_100052A2
)010466                 jmp     short loc_100103F6
```

So memcmp returns the 0 or non zero
jnz = condition being false
jz = condition being true

here we can see the comparison, if the string is not equal the function jumps to 10010468 and if the strings are equal it jumps to 100052A2 which uses socket function indicating that the string functions as a keyword that triggers network activity

```
; Attributes: bp-based frame

; int __cdecl sub_100052A2(SOCKET s)
sub_100052A2    proc near               ; CODE XR

Buffer          = byte ptr -60Ch
var_60B         = byte ptr -60Bh
Data            = byte ptr -20Ch
var_20B         = byte ptr -20Bh
cbData          = dword ptr -0Ch
Type            = dword ptr -8
hkResult        = dword ptr -4
                = dword ptr  8
```

What does the export PSLIST do?

```
0007025
0007025 ; int __stdcall PSLIST(int, int, char *Str, int)
0007025                 public PSLIST
0007025 PSLIST          proc near               ; DATA XREF: .rd
```

It calls a function

```
l00036C3
l00036C3 VersionInformation= _OSVERSIONINFOA ptr -94h
l00036C3
l00036C3                push    ebp
l00036C4                mov     ebp, esp
l00036C6                sub     esp, 94h
l00036CC                lea     eax, [ebp+VersionInformation]
l00036D2                mov     [ebp+VersionInformation.dwOSVersionInfoSize], 94h
l00036DC                push    eax             ; lpVersionInformation
l00036DD                call    ds:GetVersionExA
l00036E3                cmp     [ebp+VersionInformation.dwPlatformId], 2
l00036EA                jnz     short loc_100036FA
l00036EC                cmp     [ebp+VersionInformation.dwMajorVersion], 5
l00036F3                jb      short loc_100036FA
l00036F5                push    1
l00036F7                pop     eax
l00036F8                leave
l00036F9                retn
l00036FA ; ---------------------------------------------------------------------------
```

The function does a comparison of the operating system platform id with 2 and OS major version with 5 which is done with the help of GETVERSION API

```
Platform ID = 2      // "I'm a Windows GUI app"
Major Version = 10   // "I need at least Windows 10"
```

This is what it is needed for
Major version = 5 → Windows 2000 / XP

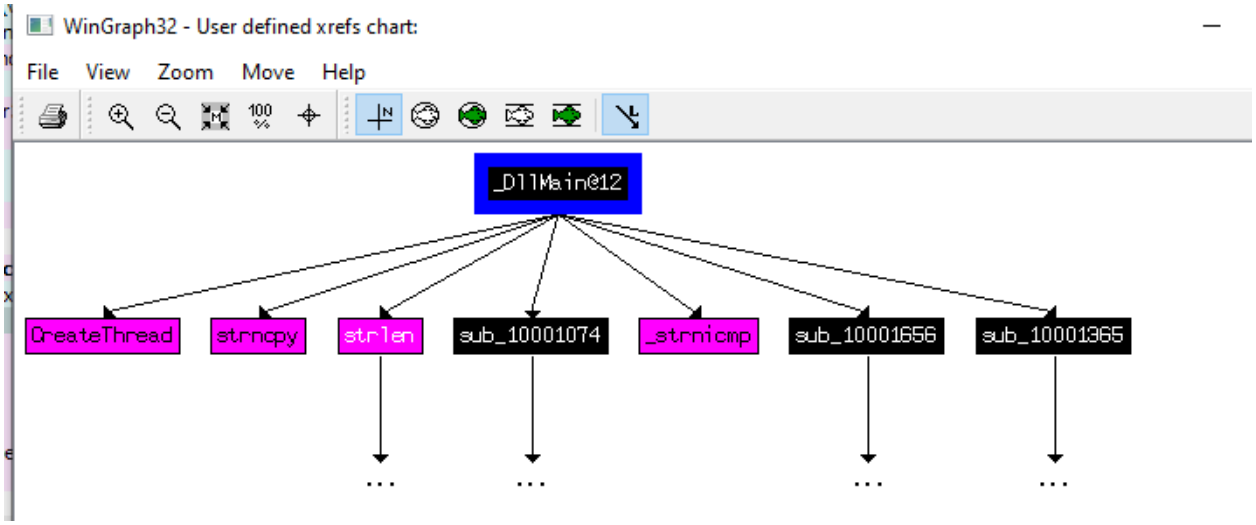
Use the graph mode to graph the cross-references from sub_10004E79. Which API functions could be called by entering this function? Based on the API functions alone, what could you rename this function?

System Default Language ID API = Functions that tell you what language Windows is using.

```
stosw
stosb
call    ds:GetSystemDefaultLangID
movzx   eax, ax
push    eax
lea     eax, [ebp+Buffer]
push    offset aLanguageId0xX ; "\r\n\r\n[Language:] id:0x%x\r\n\r\n"
push    eax             ; Buffer
call    ds:sprintf
```

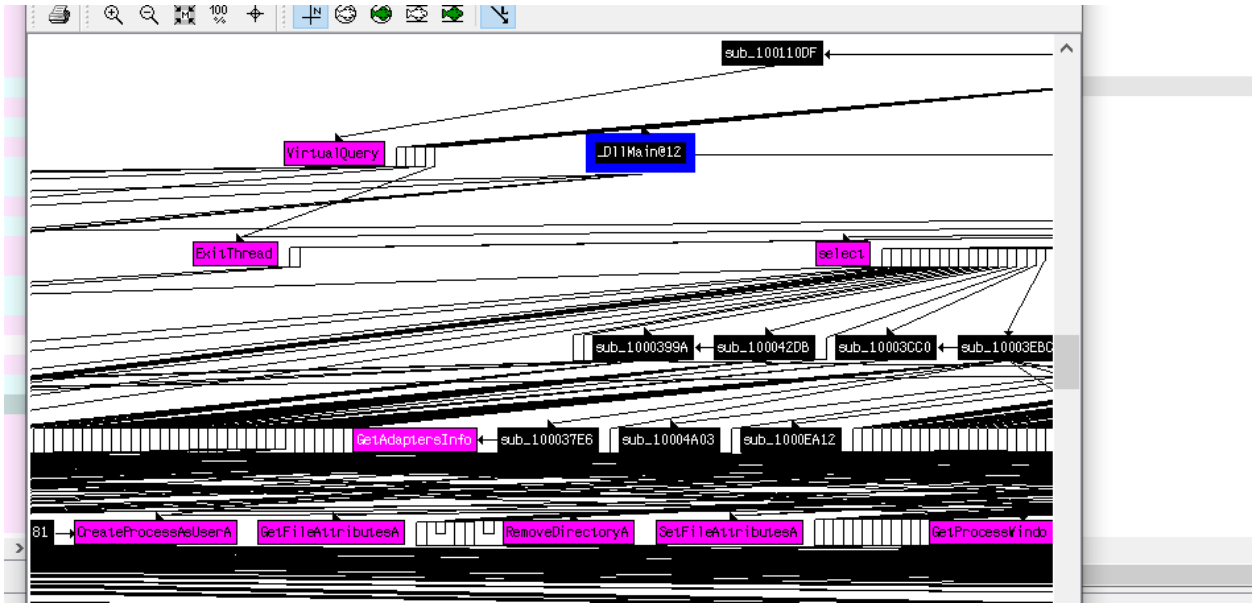
It is using this API , I can rename this function to probably WindowLanguagechecker ()

How many Windows API functions does DllMain call directly? How many at a depth of 2?



with the depth of 1 with the depth of 2 there are more than30 functions since

one calls the other.

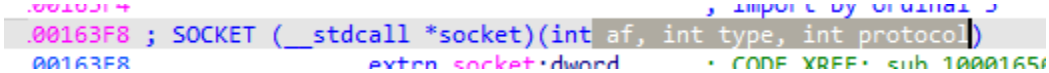


At 0x10001358, there is a call to Sleep (an API function that takes one parameter containing the number of milliseconds to sleep). Looking backward through the code, how long will the program sleep if this code executes?

```
                        ; sub_10001074+1E
mov     eax, off_10019020 ; "[This is CTI
add     eax, 13
push    eax             ; String
call    ds:atoi
imul    eax, 1000
pop     ecx
push    eax             ; dwMilliseconds
call    ds:Sleep
xor     ebp, ebp
```

first of all the offset of the string is being moved to the eax later we have eax + 13 so skipping the 13 characters then it converts the value of that string to the integer using the "**atoi**" which is later multiplied by thousand and that will the milliseconds for the malware to sleep

At 0x10001701 is a call to socket. What are the three parameters?



These are the 3 parameters being accepted by the function

Using the MSDN page for socket and the named symbolic constants func tionality in IDA Pro, can you make the parameters more meaningful? What are the parameters after you apply changes?

socket function (winsock2.h)

The **socket** function creates a socket that is bound to a specific transport serv provider.

Syntax

C++

```
SOCKET WSAAPI socket(
  [in] int af,
  [in] int type,
  [in] int protocol
);
```

One-Sentence Summary

- `af` → *How addresses look*
- `type` → *How data behaves*
- `protocol` → *Which networking engine runs the show*

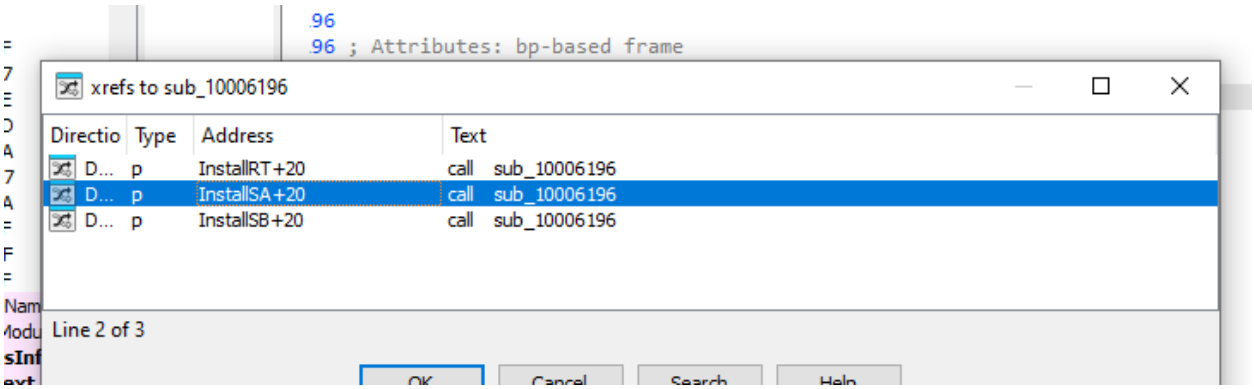
based on the information about the function we can name it network setup function
and the parameters as : IP Address type
Socket Type
Protocol

Search for usage of the in instruction (opcode 0xED). This instruction is used with a magic string VMXh to perform VMware detection. Is that in use in this malware? Using the cross-references to the function that executes the in instruction, is there further evidence of VMware detection?

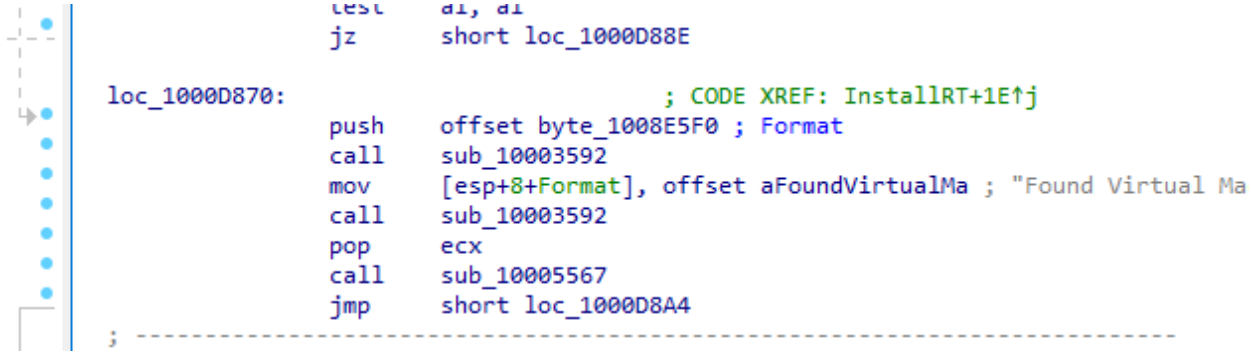
```
mov     edx, 5658h
in      eax, dx
cmp     ebx, 'VMXh'
```

yes the magic string is being used

here we can see 3 functions from the cross reference



By visiting the sub_routine 10006196 we can see the message



“Found virtual machine ,cancel install”

```
                        ; CODE XREF: InstallSB+1E↑j
push    offset byte_1008E5F0 ; Format
call    sub_10003592
mov     [esp+8+Format], offset aFoundVirtualMa ; "Found Virtual Machine,Install Cancel."
call    sub_10003592
pop     ecx
call    sub_10005567
jmp     short loc_1000E8F4
```

it is also found on other 3 sub_routines as well

although It is unknown how it is done but this part confirms that the checking of vm is being taken place

Jump your cursor to 0x1001D988. What do you find?

```
D99C                db  3Ah ; :
D99D                db  27h ; '
D99E                db  79h ; y
D99F                db  75h ; u
D9A0                db  26h ; &
D9A1                db  21h ; !
D9A2                db  27h ; '
D9A3                db  3Ch ; <
D9A4                db  3Bh ; ;
D9A5                db  32h ; 2
D9A6                db  75h ; u
D9A7                db  31h ; 1
D9A8                db  30h ; 0
D9A9                db  36h ; 6
D9AA                db  3Ah ; :
D9AB                db  31h ; 1
D9AC                db  30h ; 0
D9AD                db  31h ; 1
D9AE                db  75h ; u
D9AF                db  33h ; 3
D9B0                db  3Ah ; :
D9B1                db  27h ; '
D9B2                db  75h ; u
D9B3                db    5
D9B4                db  27h ; '
D9B5                db  34h ; 4
D9B6                db  36h ; 6
D9B7                db  21h ; !
D9B8                db  3Ch ; <
D9B9                db  36h ; 6
D9BA                db  34h ; 4
```

The data at address `0x1001D988` appears to be obfuscated or encoded rather than plain text.
This may represent encrypted configuration data, packed strings, or command-and-control related information, which is commonly used by malware to evade static analysis.

----------END----------